Ordre des branches des deux projets

Ce document présente le parcours du projet de gestion de documents. Il indique l'ordre des branches, le rôle de chacune et les étapes où les deux projets communiquent.

Les deux projets

Projet	Dossier	Rôle
Interface HTML, CSS et JavaScript	`apps/app-frontend-documents`	Affiche les pages, le formulaire et la liste des documents dans le navigateur.
Laravel et PostgreSQL	`apps/app-laravel-postgresql`	Fournit l'API, vérifie les données et enregistre les informations dans PostgreSQL.

Chaque projet possède son propre dépôt Git et ses propres branches. Changer de branche dans l'un ne change pas la branche de l'autre.

Cloner les deux projets

À faire une seule fois sur un nouveau PC : créez ou choisissez un dossier pour la formation, ouvrez-y un terminal PowerShell, puis lancez les deux commandes suivantes.

Pour récupérer l'interface sur sa première branche du jour 1 :

```
git clone --branch jour1-interface https://github.com/kamel-collab/formation-frontend-documents.git app-frontend-documents
```

Pour récupérer Laravel sur sa branche de départ :

```
git clone --branch starter https://github.com/kamel-collab/formation-laravel-special.git app-laravel-postgresql
```

Les deux dossiers sont créés côte à côte dans le dossier où le terminal est ouvert. Les commandes ne doivent pas être relancées si ces dossiers existent déjà : ouvrez directement les projets existants. Pour changer de branche ensuite, ouvrez un terminal dans le dossier du projet concerné et utilisez les commandes du tableau ci-dessous.

Parcours dans l'ordre

Dans le terminal du projet indiqué, copie la commande à côté de la branche.

Ordre	Jour	Projet	Branche et commande	Ce que contient cette étape
1	1	Interface	`jour1-interface` `git switch jour1-interface`	Interface HTML/CSS/JS, formulaire et liste fictive. Les ajouts restent temporaires dans la page.
2	1	Interface	`jour1-ressources-stockage` `git switch jour1-ressources-stockage`	Ressources locales et préférence d'affichage enregistrée dans le navigateur. Les documents restent fictifs.

Ordre	Jour	Projet	Branche et commande	Ce que contient cette étape
3	2	Laravel	`starter` `git switch starter`	Point de départ Laravel avec Docker, PostgreSQL et pgAdmin. Pas encore d'API CRUD des documents.
4	2	Laravel	`jour2-decouverte-laravel` `git switch jour2-decouverte-laravel`	Découverte de Laravel et d'une première route qui renvoie un document fictif en JSON.
5	2	Laravel	`jour2-postgresql-migrations-modeles` `git switch jour2-postgresql-migrations-modeles`	Migration et modèle `Document` pour enregistrer les informations en base.
6	2	Laravel	`jour2-api-crud-documents` `git switch jour2-api-crud-documents`	API CRUD des documents : liste, création, lecture, modification et suppression, avec validation côté serveur.
7	2	Interface	`jour2-fetch-documents` `git switch jour2-fetch-documents`	L'interface utilise Fetch pour charger et créer des documents via l'API Laravel du jour 2.
8	3	Laravel	`jour3-form-requests` `git switch jour3-form-requests`	La validation des documents est organisée dans une Form Request.
9	3	Laravel	`jour3-securite-documents` `git switch jour3-securite-documents`	API protégée avec Sanctum, documents liés à leur propriétaire et fichiers PDF privés.
10	4	Interface	`jour4-interface-api-securisee` `git switch jour4-interface-api-securisee`	Correction de la connexion à l'API protégée, des documents personnels et des PDF.
11	4	Laravel	`jour4-valkey-cache-documents` `git switch jour4-valkey-cache-documents`	Correction Valkey, cache des listes personnelles et invalidation.
12	4	Laravel	`jour4-rate-limiting-documents` `git switch jour4-rate-limiting-documents`	Limitation pédagogique des requêtes sur la liste des documents.
13	5	Laravel	`jour5-k6-performance-documents` `git switch jour5-k6-performance-documents`	Scénarios k6, comparaison du cache et observation des réponses 429.

L'ordre 6 puis 7 correspond au moment où les deux projets sont reliés. Les étapes 8 et 9 font évoluer le backend ; l'étape 10 adapte l'interface à cette API sécurisée.

Quelles branches utiliser ensemble ?

Situation	Branche interface	Branche Laravel	Résultat
Jour 1	`jour1-interface`, puis `jour1-ressources-stockage`	Pas nécessaire	L'interface fonctionne seule, avec des données fictives.
Jour 2 : liaison Fetch	`jour2-fetch-documents`	`jour2-api-crud-documents`	L'interface lit et crée des documents avec l'API et PostgreSQL, après configuration et migrations.
Jour 3 : validation	`jour2-fetch-documents`	`jour3-form-requests`	L'API reste utilisable par l'interface du jour 2 ; sa validation est organisée avec une Form Request.
Jour 3 : sécurité	`jour2-fetch-documents`	`jour3-securite-documents`	L'interface de départ n'envoie pas encore le token : tester avec Postman.
Jour 4 : interface sécurisée	`jour4-interface-api-securisee`	`jour3-securite-documents`	Connexion, documents personnels et PDF depuis l'interface.
Jour 4 : Valkey et cache	`jour4-interface-api-securisee`	`jour4-valkey-cache-documents`	Même API sécurisée avec cache des listes par utilisateur.
Jour 4 : limitation des requêtes	`jour4-interface-api-securisee`	`jour4-rate-limiting-documents`	Cache et limite de cinq lectures de liste par minute et par utilisateur.
Jour 5 : tests k6	`jour4-interface-api-securisee`	`jour5-k6-performance-documents`	Mesure locale de l'API protégée avec Docker et k6.

Fichiers : l'interface du jour 2 ne téléverse pas de fichier. L'interface du jour 4 utilise les routes de l'API sécurisée pour envoyer et télécharger les PDF.

Guides et corrections

Les guides se trouvent dans le projet concerné. Pour un TD, partez de la branche indiquée ; la branche suivante permet de consulter la correction.

Exercice	Projet et branche de départ	Guide	Branche de correction
Démarrer Laravel et Docker	Laravel : `starter`	`TD-Demarrage-Docker.md`	`jour2-decouverte-laravel` pour la première route d'exemple
Créer la migration et le modèle	Laravel : `jour2-decouverte-laravel`	`TD-Documents-Migration-Modele.md`	`jour2-postgresql-migrations-modeles`
Construire l'API CRUD	Laravel : `jour2-postgresql-migrations-modeles`	`TD-API-CRUD-Documents.md`	`jour2-api-crud-documents`
Ressources et stockage navigateur	Interface : `jour1-interface`	`TD-Ressources-et-stockage.md`	`jour1-ressources-stockage`
Relier l'interface avec Fetch	Interface : `jour1-ressources-stockage` ; API Laravel : `jour2-api-crud-documents`	`TD-Fetch-Documents.md`	Interface : `jour2-fetch-documents`

Exercice	Projet et branche de départ	Guide	Branche de correction
Organiser la validation	Laravel : `jour2-api-crud-documents`	`TD-Form-Request-Documents.md`	`jour3-form-requests`
Sécuriser l'API et les fichiers	Laravel : `jour3-form-requests`	`TD-Securite-Documents.md`	`jour3-securite-documents`
Connecter l'interface à l'API sécurisée	Interface : `jour2-fetch-documents` ; API Laravel : `jour3-securite-documents`	`TD-Interface-API-Securisee.md`	Interface : `jour4-interface-api-securisee`
Connecter Valkey, mettre en cache et invalider	Laravel : `jour3-securite-documents`	`TD-Valkey-Cache-Documents.md`	Laravel : `jour4-valkey-cache-documents`
Limiter les requêtes et observer 429	Laravel : `jour4-valkey-cache-documents`	`TD-Valkey-Cache-Documents.md`, partie 4	Laravel : `jour4-rate-limiting-documents`
Tester l'API avec k6	Laravel : `jour4-rate-limiting-documents`	`TD-k6-API-Documents.md`	Laravel : `jour5-k6-performance-documents`

Le guide de sécurité propose un seul TD en trois étapes : authentification, propriété des documents, puis PDF privé. Chaque étape se vérifie dans Postman avant de continuer. Dans pgAdmin, les étudiants peuvent aussi observer les données de formation.

Pour les essais actuels

- Laravel : `jour3-securite-documents` pour démarrer le TD Valkey, `jour4-valkey-cache-documents` pour sa correction et la partie rate limiting, ou `jour4-rate-limiting-documents` pour la correction finale. Suivez le guide pour installer Predis et configurer `.env` avant d'utiliser Valkey.
- Interface : `jour2-fetch-documents` pour réaliser le TD du jour 4, ou `jour4-interface-api-securisee` pour consulter sa correction.
- API à tester avec Postman : `http://localhost:8088/api`.
- pgAdmin : `http://localhost:5050`.
- Le TD du jour 4 réalise la liaison de l'interface à l'API sécurisée.

Les mises à jour Postman et les branches du jour 4 ont été publiées sur GitHub. Pour les récupérer, exécutez `git fetch origin` dans chaque dépôt, puis vérifiez les branches disponibles avant de changer de branche. Préservez votre travail local.

Vérifier et changer de branche

Dans le terminal de **chaque projet**, vérifiez votre branche et votre travail :

```
git branch --show-current
git status
```

Après avoir sauvegardé votre travail, changez de branche dans le projet concerné :

```
git switch nom-de-la-branche
```

Par exemple, pour consulter la correction de sécurité, exécutez `git switch jour3-securite-documents` dans le dossier Laravel. Un changement de branche modifie les fichiers du projet, mais ne remet pas automatiquement la base de données à son état précédent. Suivez les indications du TD avant d'exécuter une migration ou un seeder.

Si Git refuse le changement de branche

Git peut afficher « Your local changes would be overwritten » lorsque des fichiers ont été modifiés. Placez-vous à la **racine du projet concerné**, puis vérifiez les fichiers avec `git status`. Choisissez ensuite l'une des deux solutions suivantes.

Solution 1 : conserver temporairement les modifications

Pour mettre de côté les modifications des fichiers suivis et les nouveaux fichiers non suivis, puis changer de branche :

```
git stash push -u -m "sauvegarde avant changement de branche"
git switch nom-de-la-branche
```

Les fichiers ignorés par Git ne sont pas sauvegardés par cette commande. Pour retrouver les modifications plus tard, revenez d'abord sur la branche de départ, puis affichez la liste :

```
git stash list
```

Repérez la sauvegarde par son nom, puis appliquez-la. Dans l'exemple ci-dessous, remplacez `stash@{0}` si votre sauvegarde porte un autre numéro :

```
git stash apply 'stash@{0}'
```

`apply` réapplique les modifications tout en conservant la sauvegarde dans le stash.

Solution 2 : annuler les modifications

Attention : ces commandes effacent les modifications non enregistrées dans un commit. Utilisez-les seulement si vous êtes sûr de ne plus en avoir besoin.

Pour annuler les modifications d'un seul fichier, par exemple `index.html` :

```
git restore --source=HEAD --staged --worktree -- index.html
git switch nom-de-la-branche
```

Pour annuler les modifications de **tous les fichiers suivis**, y compris ceux déjà ajoutés avec `git add` :

```
git restore --source=HEAD --staged --worktree .
git switch nom-de-la-branche
```

Ces commandes ne suppriment pas les nouveaux fichiers non suivis. Vérifiez-les avec `git status` ; ils peuvent aussi empêcher le changement de branche s'ils portent le même nom qu'un fichier de la branche cible.